

Stock Portfolio Manager

Complete Project and Interview Guide

Contents

Stock Portfolio Manager — Complete Project & Interview Guide	1
Table of Contents	1
1. Project at a Glance	2
2. Technology Stack	2
3. High-Level Design (HLD)	3
4. Low-Level Design (LLD) — Backend	5
5. Low-Level Design (LLD) — Frontend	6
6. Data Model & Database	8
7. API Reference	9
8. Security Deep Dive	11
9. Key Algorithms Explained	12
10. Glossary of Terms & Definitions	13
11. Interview Questions — Conceptual / Definitions	14
12. Interview Questions — How This Project Works	15
13. Interview Questions — How Things Connect	17
14. Interview Questions — Why This, Not That (Design Decisions)	18
15. Interview Questions — Situational (“What would you do if...”)	19
16. Interview Questions — Coding / Hands-on Drill-downs	21
17. Interview Questions — Behavioural / Project-story	23
18. Known Limitations & How to Talk About Them	23
19. Quick Revision Cheat-Sheet	24

Stock Portfolio Manager — Complete Project & Interview Guide

A full walkthrough of the project from high-level architecture down to the smallest implementation detail, plus a large bank of interview questions (with answers) tuned for a fresher / 0-2 years experience candidate.

Table of Contents

1. Project at a Glance
2. Technology Stack
3. High-Level Design (HLD)
4. Low-Level Design (LLD) — Backend
5. Low-Level Design (LLD) — Frontend
6. Data Model & Database
7. API Reference
8. Security Deep Dive
9. Key Algorithms Explained
10. Glossary of Terms & Definitions

11. Interview Questions — Conceptual / Definitions
 12. Interview Questions — How This Project Works
 13. Interview Questions — How Things Connect
 14. Interview Questions — Why This, Not That (Design Decisions)
 15. Interview Questions — Situational (“What would you do if...”)
 16. Interview Questions — Coding / Hands-on Drill-downs
 17. Interview Questions — Behavioural / Project-story
 18. Known Limitations & How to Talk About Them
 19. Quick Revision Cheat-Sheet
-

1. Project at a Glance

Stock Portfolio Manager is a full-stack web application that lets a registered user manage a *simulated* stock portfolio using virtual money. Every new user starts with **\$100,000 of virtual cash**. They can search for real stock symbols, view live prices and historical price charts, buy and sell shares, watch their holdings’ profit/loss update against live market prices, and ask an AI advisor (Google Gemini) to analyse their portfolio.

It is a “paper trading” / portfolio-tracking app: no real money and no real brokerage is involved, but the **prices are real**, pulled from live market-data APIs (Finnhub and Twelve Data).

What the user can do:

- Register and log in (JWT-based authentication).
- Search for stocks by symbol/company name.
- See a live quote and an interactive historical price chart (1D / 1W / 1M / 1Y / ALL).
- Buy shares (cash is deducted) and sell shares (cash is credited).
- View a dashboard with cash balance, total portfolio value, total profit/loss, and holdings count.
- View a detailed, paginated, sortable portfolio table with per-stock profit/loss.
- Get an AI-generated portfolio analysis (scores, risk level, strengths, weaknesses, recommendations).
- Toggle light/dark theme.

The system is split into two independently-deployable applications:

- stock-backend — a **Spring Boot (Java 21)** REST API that owns all business logic, the database, security, and all external API calls.
- stock-frontend — a **React 19 + Vite** single-page application (SPA) that talks to the backend over HTTP/JSON.

A **PostgreSQL** database persists users, holdings, transactions, latest prices, and a cache of historical price series.

2. Technology Stack

Backend (stock-backend)

Concern	Choice
Language / Runtime	Java 21
Framework	Spring Boot (Web MVC)
Security	Spring Security + JWT (JJWT 0.12.x)
Persistence	Spring Data JPA / Hibernate
Database	PostgreSQL
HTTP client (outbound)	Spring RestClient (synchronous)
Validation	Jakarta Bean Validation

Concern	Choice
Build tool	Maven
Boilerplate reduction	Lombok
Scheduling	Spring @Scheduled

Frontend (stock-frontend)

Concern	Choice
Library	React 19
Build tool / dev server	Vite 7
Routing	React Router 7
HTTP client	Axios
Charts	Recharts 3
Styling	Tailwind CSS 4 (CSS-first config)
State management	React Context API + hooks (no Redux)

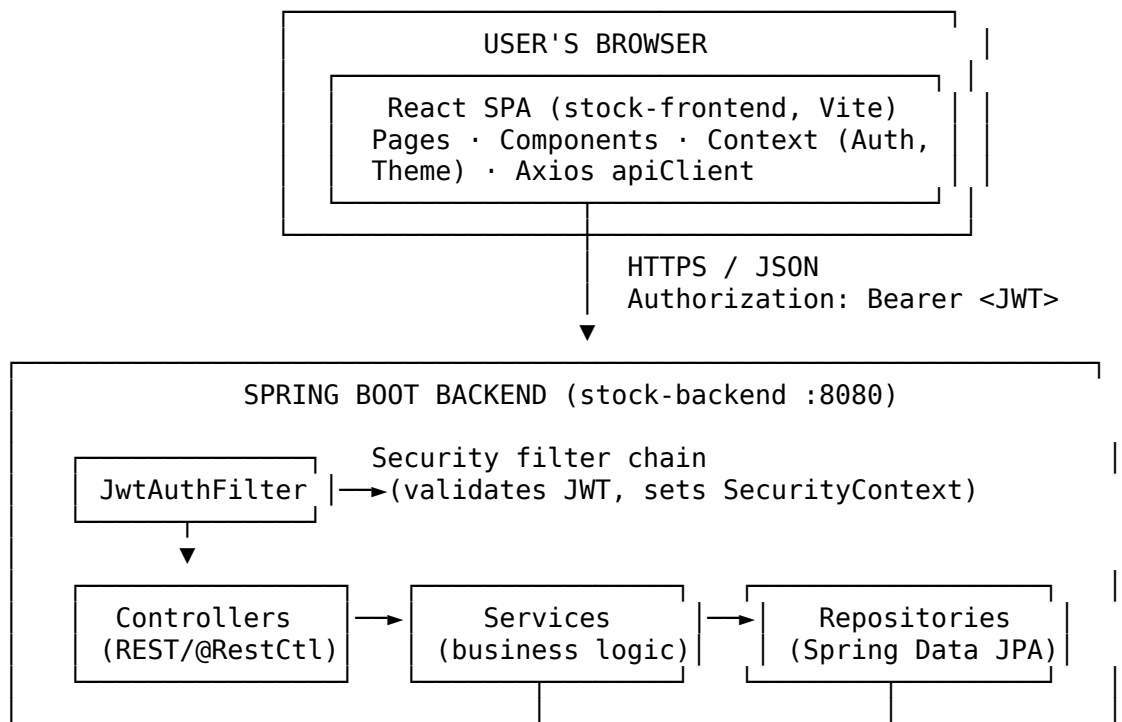
External services

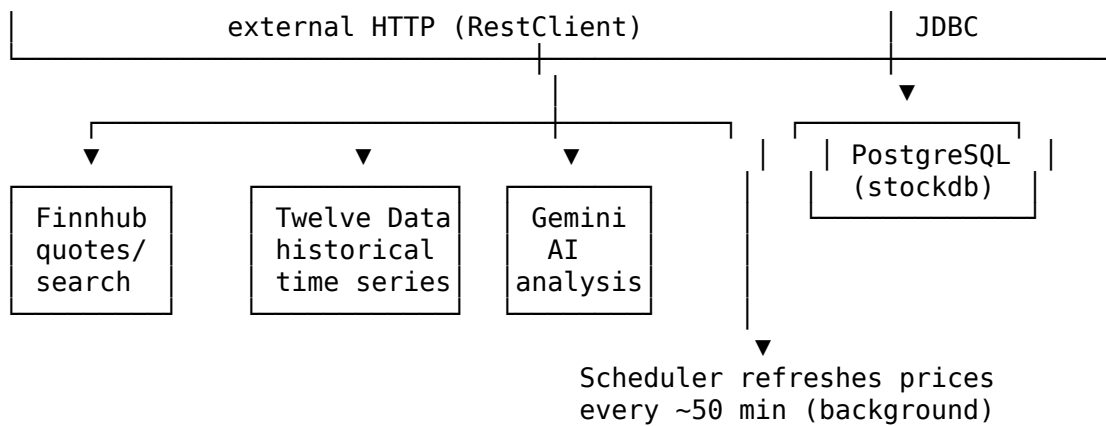
- **Finnhub** — live stock quotes and symbol search.
- **Twelve Data** — historical time-series (for the price charts).
- **Google Gemini** (gemini-2.5-flash) — AI portfolio analysis.
- **API Ninjas** — company logos (called directly from the browser).

3. High-Level Design (HLD)

The High-Level Design describes the big building blocks and how they talk to each other, without diving into individual classes or methods.

3.1 System architecture





3.2 The layered (n-tier) backend architecture

The backend follows the classic **Controller → Service → Repository** layering:

- **Controller layer** — exposes REST endpoints, handles HTTP concerns (status codes, request/response bodies, validation triggers). It never contains business logic.
- **Service layer** — the “brain”. Holds all business rules: trade logic, valuation maths, calling external APIs, caching, building the AI prompt. Transaction boundaries (`@Transactional`) live here.
- **Repository layer** — Spring Data JPA interfaces that talk to PostgreSQL. No hand-written SQL for CRUD; a few custom JPQL queries for valuation.
- **Cross-cutting layers** — Security (JWT filter, config), Exception handling (`@RestControllerAdvice`), Configuration (beans), Scheduling.

This separation means each layer has one responsibility, can be tested in isolation, and can change independently (e.g. swap the DB without touching controllers).

3.3 Request lifecycle (end-to-end, “buy a stock”)

1. User clicks **Buy** in the React TradeForm.
2. `portfolioService.buyStock()` (frontend) calls Axios → POST `/api/portfolio/buy` (base URL `http://localhost:8080`) with `{ symbol, quantity }`.
3. The Axios **request interceptor** attaches `Authorization: Bearer <JWT>` (token read from `localStorage`).
4. On the backend, the **JwtAuthenticationFilter** intercepts the request, validates the token, loads the user, and puts an `Authentication` into the `SecurityContext`.
5. `PortfolioController.buy()` reads the authenticated user’s email and calls `PortfolioService.buyStock(email, request)`.
6. The service fetches a **live price from Finnhub**, checks the user has enough cash, updates/creates the holding, recomputes the weighted-average buy price, deducts cash, and records a `Transaction` — all inside one DB transaction.
7. A `PortfolioHoldingResponse` DTO is returned as JSON.
8. The frontend shows a success message and refreshes the portfolio summary.

3.4 Two data-freshness strategies

- **On-demand (synchronous):** buying/selling and viewing a quote fetch the price *live* at request time.
- **Background (scheduled):** a `@Scheduled` job wakes up every ~50 minutes and refreshes the latest price for every symbol the system already tracks, so portfolio valuations stay reasonably fresh even without user activity.
- **Cached:** historical chart data is cached in the database with a per-range TTL (time-to-live), so repeat chart views don’t hammer the Twelve Data API.

4. Low-Level Design (LLD) — Backend

Base package: `com.stock.stockbackend`. Below is the package-by-package breakdown.

4.1 Application bootstrap & configuration

- **StockBackendApplication** — the `@SpringBootApplication` entry point. It has a static block that forces the whole JVM to UTC (`TimeZone.setDefault(UTC)`) so all timestamps are consistent regardless of server locale.
- **SchedulingConfig** — `@Configuration @EnableScheduling`; just switches on Spring's scheduler so `@Scheduled` methods run.
- **StockApiConfig** — defines three `RestClient` beans, one per external API:
 - `finnhubRestClient` — base URL `https://finnhub.io/api/v1`, sends the API key in the `X-Finnhub-Token` header, 5s connect / 10s read timeouts.
 - `twelveDataRestClient` — base URL `https://api.twelvedata.com`, key passed **per request as a query param**, same timeouts.
 - `geminirestClient` — base URL `https://generativelanguage.googleapis.com`, `Content-Type: application/json`, longer **60s read timeout** (AI responses are slow).
- **SecurityConfig** — the Spring Security filter chain (covered in §8).

Config values live in `application.properties`: DB connection, JPA settings (`ddl-auto=update`, `show-sql=true`), JWT secret & 24h expiry, the three API base URLs & keys, timeouts, and the scheduler's delays. Secrets are read via `${ENV_VAR:default}` placeholders.

4.2 Entity layer (entity, enums)

All entities extend **BaseEntity** (`@MappedSuperclass`) which provides:

- Long `id` — identity-generated primary key.
- Instant `createdAt` — set once via `@CreationTimestamp`.
- Instant `updatedAt` — refreshed via `@UpdateTimestamp`.

Entities: `User`, `Portfolio`, `Transaction`, `StockPrice`, `StockHistoryCache`. Enums: `Role` (`USER`, `ADMIN`), `TransactionType` (`BUY`, `SELL`), `HistoricalRange` (`1D`/`1W`/`1M`/`1Y`/`ALL`, each carrying an API interval + output size). Full field details are in §6.

4.3 Repository layer (repository)

Spring Data JPA interfaces (each extends `JpaRepository<Entity, Long>`):

- **UserRepository** — `findByEmail`, `existsByEmail`.
- **PortfolioRepository** — the most interesting one. Besides finders like `findByUserEmailAndStockSymbol`, it has two **custom JPQL queries**:
 - `findActiveHoldingsForValuation(...)` — a paginated query that `LEFT JOINS` `StockPrice` on `symbol` and computes holding value & profit/loss in SQL, with a dynamic `ORDER BY` built from CASE expressions (so sorting happens in the database).
 - `calculateValuationTotals(...)` — an aggregate query returning grand-total portfolio value and total P/L, mapped to a **projection interface** `PortfolioValuationTotals`.
- **StockPriceRepository** — `findByStockSymbol`, `findByStockSymbolIn` (batch), `findTopByStockSymbolOrderByPriceTimestampDesc`.
- **StockHistoryCacheRepository** — `findByStockSymbolAndTimeRange`.
- **TransactionRepository** — plain CRUD.

4.4 DTO layer (dto)

DTOs are Java **records** (immutable data carriers) used so the API contract never exposes entities directly. External-response DTOs use `@JsonIgnoreProperties(ignoreUnknown = true)` so extra fields from third-party APIs don't break deserialization.

Key request DTOs: RegisterRequest, LoginRequest, PortfolioTradeRequest (all Bean-Validated). Key response DTOs: AuthResponse, PortfolioHoldingResponse, PortfolioValuationResponse (paginated), StockPriceResponse, StockHistoryPointResponse, StockSearchResponse, AiAdvisorResponse. External-mapping DTOs: FinnhubQuoteResponse, TwelveDataTimeSeriesResponse, GeminiModel.

4.5 Controller layer (controller)

Thin @RestController that map HTTP to service calls. They obtain the current user from the injected Authentication object (authentication.getName() returns the email, which is the JWT subject). Endpoints are catalogued in §7.

4.6 Service layer (service)

- **AuthService** — register (hash password with BCrypt, save user, issue JWT) and login (delegate to Spring's AuthenticationManager, then issue JWT).
- **PortfolioService** — the money engine: buyStock, sellStock, getPortfolio (paginated valuation), removeHolding. All BigDecimal maths at scale 4, HALF_UP rounding. Detailed in §9.
- **StockApiService** — the raw outbound HTTP calls to Finnhub / Twelve Data, including error-status mapping (429 → rate-limit exception, other errors → StockApiException).
- **StockPriceService** — orchestrates prices: fetch-and-save latest quote, list tracked symbols (for the scheduler), and historical prices with the DB-backed TTL cache and label formatting.
- **AiAdvisorService** — builds the Gemini prompt from the user's holdings, calls the Gemini API, and parses the JSON reply into AiAdvisorResponse.

4.7 Security layer (security)

JwtService (create/validate tokens), JwtAuthenticationFilter (per-request token check), CustomUserDetailsService (load user by email). Detailed in §8.

4.8 Scheduling (scheduler)

StockPriceUpdateScheduler — one @Scheduled method with initialDelay = 10 min and fixedDelay = 50 min. It reads all tracked symbols and refreshes each one's price, catching exceptions **per symbol** so one bad ticker never aborts the whole batch. Because it uses fixedDelay (not fixedRate), runs never overlap.

4.9 Exception handling (exception)

A single @RestControllerAdvice (GlobalExceptionHandler) maps each custom exception to an HTTP status and a consistent ApiError body { status, message, timestamp }. Validation failures return a field → message map. See §7.4.

5. Low-Level Design (LLD) — Frontend

Root: stock-frontend/. It is a Vite-powered React 19 SPA.

5.1 Bootstrap & provider nesting

- main.jsx mounts <App /> inside <StrictMode>.
- App.jsx simply renders <AppRoutes />.
- Providers nest as: **BrowserRouter** → **ThemeProvider** → **AuthProvider** → **Routes**. Theme is the outermost app-level provider; auth wraps the routed content.

5.2 Routing (routes/, layouts/)

Every route is a child of one layout route (<AppLayout />):

Path	Component	Access
/	redirect → /dashboard	—
/login	LoginPage	public
/register	RegisterPage	public
/dashboard	DashboardPage	protected
/portfolio	PortfolioPage	protected
/buy-sell	BuySellPage	protected
*	NotFoundPage	public

- **ProtectedRoute** reads `isAuthenticated` from `useAuth()`. If not logged in it redirects to `/login`, saving the attempted location in `state.from` so the user is sent back there after login. Otherwise it renders `<Outlet />`.
- **AppLayout** renders two shells: a simple top nav for logged-out users, and a sidebar + header shell (with theme toggle and cash-balance chip) for logged-in users.

5.3 State management (context/, hooks/)

No Redux — just the **Context API**:

- **Auth**: `AuthContext` + `AuthProvider`. Holds the JWT token (initialised lazily from `localStorage`) and the user's `cashBalance`. Exposes `login`, `register`, `logout`, `isAuthenticated`, `cashBalance`, `setCashBalance`. When a token exists, an effect fetches the portfolio once to populate the cash balance shown in the header. `useAuth()` is the accessor hook.
- **Theme**: `ThemeContext` + `ThemeProvider`. Persists the chosen theme in `localStorage` (falling back to the OS preference), and toggles a `.dark` class on `<html>` (Tailwind v4 class-based dark mode). `useTheme()` exposes `{ theme, isDark, toggleTheme }`.

5.4 Service / API layer (services/, utils/)

- **apiClient.js** — a single Axios instance with `baseUrl: http://localhost:8080`. A **request interceptor** reads the JWT from `localStorage` and adds `Authorization: Bearer <token>` to every call. (There is no response interceptor / auto-refresh — a deliberately simple design.)
- **authService.js** — `loginUser` (POST `/api/auth/login`), `registerUser` (POST `/api/auth/register`).
- **portfolioService.js** — `getPortfolio`, `buyStock`, `sellStock`, `getStockHistory`, `getStockQuote`, `searchStocks`, `analyzePortfolio` (each maps to a backend endpoint).
- **logoService.js** — fetches company logos directly from API Ninjas (raw fetch, not the Axios client) and caches them in `localStorage`.
- **tokenStorage.js** — `getToken` / `setToken` / `removeToken`; the JWT lives in `localStorage` under `stock_portfolio_token`.
- **formatters.js** — `formatCurrency`, `formatNumber`, `getErrorMessage`.

5.5 Pages (pages/)

- **LoginPage / RegisterPage** — controlled forms that call `login/register`, then navigate (login honours the saved from location).
- **DashboardPage** — four metric cards (cash, total value, total P/L, holdings count), a mini top-holdings list, a placeholder market summary, and the **AI Advisor** button/modal.
- **PortfolioPage** — paginated, sortable holdings table plus lazy-loaded chart cards (watchlist chart + allocation breakdown) using `React.lazy` + `Suspense`.
- **BuySellPage** — the richest page: symbol search with **300 ms debounced autocomplete**, a live quote, a historical price `AreaChart` with a range selector, and two `TradeForms` (buy / sell).

5.6 Components (components/)

DashboardCard (metric tile), PaginationControls, AuthFormShell, StockLogo (real logo with initial-badge fallback), PortfolioBreakdownCard (donut allocation chart), PortfolioOverviewMini, WatchlistChartCard (with an in-memory per-symbol:range cache), TradeForm (symbol locked, quantity editable, client-side validation), PortfolioTable (client-side search + sort, per-row P/L %), and AiAdvisorModal (opens → calls /api/portfolio/analyze → renders scores, risk, strengths/weaknesses, per-asset recommendations).

5.7 Notable frontend patterns

Class-based dark mode; server-side pagination; debounced autocomplete; code-splitting via lazy chart cards; in-memory + localStorage caching for logos and chart history; a request-dedup map to avoid duplicate logo fetches.

6. Data Model & Database

PostgreSQL database stockdb. All money columns are DECIMAL(19,4) and handled as BigDecimal in Java (never double/float). All timestamps are stored in UTC.

6.1 Tables & key columns

app_users (entity User) — unique index on email.

Column	Type	Notes
id	bigint PK	identity
full_name	varchar(100)	not null
email	varchar(150)	not null, unique
password_hash	varchar(255)	BCrypt hash, never plaintext
role	varchar(20)	USER / ADMIN
cash_balance	decimal(19,4)	defaults to 100000.00
created_at / updated_at	timestamp	auditing

portfolios (entity Portfolio) — one row per **(user, symbol)** (unique constraint), indexed on user_id and stock_symbol.

Column	Type	Notes
id	bigint PK	
name / description	varchar	holding metadata
stock_symbol	varchar(20)	e.g. AAPL
quantity	decimal(19,4)	shares held
average_buy_price	decimal(19,4)	weighted average cost
active	boolean	soft-delete flag
user_id	bigint FK → app_users	ManyToOne

portfolio_transactions (entity Transaction) — an audit log of every trade.

Column	Type	Notes
id	bigint PK	
portfolio_id	bigint FK → portfolios	ManyToOne
stock_symbol	varchar(20)	
type	varchar(10)	BUY / SELL

Column	Type	Notes
quantity	decimal(19,4)	
price_per_share	decimal(19,4)	execution price
transaction_date	date	past-or-present

stock_prices (entity StockPrice) — latest known price per symbol (unique on symbol).

Column	Type	Notes
id	bigint PK	
stock_symbol	varchar(20)	unique
price	decimal(19,4)	
price_timestamp	timestamp	from the quote's epoch

stock_history_cache (entity StockHistoryCache) — cached chart series, unique on (**symbol**, **time_range**).

Column	Type	Notes
id	bigint PK	
stock_symbol	varchar(20)	
time_range	varchar(10)	1D/1W/1M/1Y/ALL
data_json	text	serialized list of points
updated_at	timestamp	drives TTL freshness

6.2 Relationships (ER overview)

app_users (1) —< (N) portfolios (1) —< (N) portfolio_transactions
 └─ cash_balance lives on the user

stock_prices : keyed by symbol, joined to portfolios by symbol string
 stock_history_cache : keyed by (symbol, range)

- A User **has many** Portfolio rows (@OneToMany, cascade all, orphan removal).
- A Portfolio **has many** Transaction rows.
- StockPrice and StockHistoryCache are **not** JPA relationships to Portfolio; they're joined by the symbol string in a custom query (a deliberate decoupling — prices are shared across all users).

6.3 Important design choices in the schema

- **Virtual cash on the user row** — the whole “wallet” is a single cash_balance column; there's no separate account/ledger table.
- **One holding row per symbol** — buying more of a stock you already own updates the existing row (and its average price) rather than inserting a new one.
- **Soft deletes** — selling everything or removing a holding sets active = false instead of deleting the row, preserving history.
- **Shared price tables** — stock_prices is global, so every user's valuation of AAPL uses the same latest price.

7. API Reference

Base URL (dev): <http://localhost:8080>. All non-auth endpoints require Authorization: Bearer <JWT>.

7.1 Auth (/api/auth) — public

Method	Path	Body	Response	Notes
POST	/api/auth/register	RegisterRequest {fullName, email, password}	201 AuthResponse {token, tokenType, email, role}	password 8-72 chars; 409 if email taken
POST	/api/auth/login	LoginRequest {email, password}	200 AuthResponse	401 on bad credentials

7.2 Portfolio (/api/portfolio) — protected

Method	Path	Body / Params	Response
POST	/api/portfolio/buy	PortfolioTradeRequest {symbol, quantity}	200 PortfolioHold- ingResponse
POST	/api/portfolio/sell	PortfolioTradeRequest {symbol, quantity}	200 PortfolioHold- ingResponse
GET	/api/portfolio	?symbol=&page=&size=&sort= (default size 10, sort symbol asc)	200 PortfolioValu- ationResponse (paginated)
DELETE	/api/portfolio/{id}	path id	204 No Content (soft delete)
GET	/api/portfolio/analyze		200 AiAdvisorResponse

PortfolioValuationResponse = { cashBalance, totalPortfolioValue, totalProfitLoss, page, size, totalElements, totalPages, holdings[] }, where each holding = { symbol, quantity, averageBuyPrice, currentPrice, holdingValue, profitLoss }.

7.3 Stocks (/api/stocks) — protected

Method	Path	Params	Response
GET	/api/stocks/search	?q=	StockSearchResponse {count, result[]}
GET	/api/stocks/{symbol}	—	StockPriceResponse {symbol, price, lastUpdated} (fetches live + saves)
GET	/api/stocks/history/{symbol}	?symbol={symbol}&page={page}&size={size}	List<StockHistoryPointResponse {time, price}> (cached)

7.4 Error responses

All handled by GlobalExceptionHandler returning ApiError {status, message, timestamp}:

Exception	HTTP status
EmailAlreadyExistsException	409 Conflict

Exception	HTTP status
BadCredentialsException	401 Unauthorized
PortfolioHoldingNotFoundException / StockSymbolNotFoundException	404 Not Found
InsufficientCashBalanceException / InsufficientStockQuantityException	400 Bad Request
InvalidStockSymbolException / InvalidPortfolioSortException	400 Bad Request
StockApiRateLimitException	429 Too Many Requests
StockApiException	502 Bad Gateway
MethodArgumentNotValidException (validation)	400 + {field: message} map

8. Security Deep Dive

Authentication is **stateless JWT** (no server-side session). The flow:

8.1 Registration / login → token issue

1. register: check email not taken → hash password with **BCrypt** → save User → generate JWT → return it.
2. login: `AuthenticationManager.authenticate(...)` verifies email + BCrypt password (via `DaoAuthenticationProvider` + `CustomUserDetailsService`) → on success generate JWT.

8.2 JwtService (JJWT 0.12.x, HMAC-SHA256)

- `generateToken(userDetails)` — subject = user email, `issuedAt` = now, `expiration` = now + 24h, signed with a key derived from the secret (`Keys.hmacShaKeyFor(secret bytes)`).
- **No custom claims** — the token carries only the subject (email). Roles/authorities are re-loaded from the DB on every request (fresher-friendly point: “the token stays small and roles can’t go stale”).
- `isTokenValid` — email matches the loaded user **and** token not expired.

8.3 JwtAuthenticationFilter (extends OncePerRequestFilter)

On each request: read the Authorization header → if it starts with Bearer, extract the token → `extractEmail` → load `UserDetails` → validate → build a `UsernamePasswordAuthenticationToken` and set it into the `SecurityContextHolder`. If the token is bad/expired it returns 401 `{"message": "Invalid or expired token"}`. Requests without a token simply continue as anonymous (and get blocked later if the endpoint needs auth).

8.4 SecurityConfig

- **Stateless** session policy (`SessionCreationPolicy.STATELESS`) — no `JSESSIONID`.
- **CSRF disabled** — safe here because auth is a bearer token in a header, not a cookie.
- Route rules: `/api/auth/**` and `/error` are `permitAll`; `/api/admin/**` needs `ROLE_ADMIN`; everything else needs authentication.
- The JWT filter runs **before** `UsernamePasswordAuthenticationFilter`.
- **CORS** allows the Vite dev origins (`localhost:5173`, `127.0.0.1:5173`), the standard verbs, and the Authorization header.
- Password hashing: `BCryptPasswordEncoder`.
- Custom `authenticationEntryPoint` (401) and `accessDeniedHandler` (403) return JSON messages.

8.5 Security notes to be honest about in an interview

- The JWT is stored in `localStorage` on the frontend → readable by JavaScript, so it's exposed to XSS. Alternatives: `httpOnly` cookies. (See §18.)
- Default secret and API keys have fallback values in `application.properties` — fine for local dev, must be real env vars/secrets in production.
- No refresh-token mechanism; when the 24h token expires the user must log in again.

9. Key Algorithms Explained

9.1 Buying a stock (`PortfolioService.buyStock`)

1. Normalise the symbol (trim + uppercase).
2. Fetch a LIVE market price from Finnhub (and persist it as the latest price).
3. $\text{cost} = \text{quantity} \times \text{marketPrice}$.
4. If $\text{user.cashBalance} < \text{cost}$ → throw `InsufficientCashBalanceException`.
5. Find the existing holding for (user, symbol); if none, create one.
6. $\text{newQuantity} = \text{oldQuantity} + \text{buyQuantity}$.
7. $\text{newAverageBuyPrice} = \text{weighted average}$ (see 9.3).
8. Deduct cost from `cashBalance`; save the user.
9. Save the holding (`active = true`).
10. Record a BUY Transaction at `marketPrice`.

Everything happens inside one `@Transactional` method, so if any step fails the whole trade rolls back (no half-completed trades).

9.2 Selling a stock (`PortfolioService.sellStock`)

1. Normalise symbol; load the ACTIVE holding (else `PortfolioHoldingNotFoundException`).
2. If $\text{holding.quantity} < \text{sellQuantity}$ → `InsufficientStockQuantityException`.
3. Fetch live market price.
4. $\text{remaining} = \text{quantity} - \text{sellQuantity}$.
5. If $\text{remaining} == 0$ → `active = false`, $\text{averageBuyPrice} = 0$ (position closed).
6. $\text{earnings} = \text{sellQuantity} \times \text{marketPrice}$; add to `cashBalance`; save user.
7. Save holding; record a SELL Transaction.

Note: on a **partial** sell the average buy price is *not* recomputed (it represents cost basis of remaining shares, which is unchanged). Realised profit shows up implicitly as increased cash, not stored separately.

9.3 Weighted-average buy price

$$\text{newAvg} = (\text{oldQty} \times \text{oldAvg} + \text{buyQty} \times \text{buyPrice}) / (\text{oldQty} + \text{buyQty})$$

Scaled to 4 decimals, `HALF_UP`. This is how the “average cost per share” stays correct across multiple buys at different prices.

Example: own 10 @ \$100 (cost \$1000). Buy 10 @ \$120 (cost \$1200). New avg = $\$2200 / 20 = \mathbf{\$110}$.

9.4 Portfolio valuation (done in SQL)

For the paginated portfolio view, the database itself computes:

```
holdingValue = quantity × COALESCE(livePrice, averageBuyPrice)
profitLoss   = (COALESCE(livePrice, averageBuyPrice) – averageBuyPrice) × quantity
```

`COALESCE` means: if there's no live price yet, fall back to the average buy price (so P/L reads 0 rather than crashing). Sorting (by symbol/quantity/value/P&L) and the grand totals are also computed in SQL, so the app only ships one page of rows over the wire.

9.5 Historical-price caching (StockPriceService.getHistoricalPrices)

1. Look up cache row for (symbol, range).
2. If it exists AND (updatedAt + TTL) > now → deserialize JSON and RETURN (cache hit).
3. Otherwise call Twelve Data, map to points, serialize to JSON, upsert the cache row.

TTLs by range: 1D → 5 min, 1W/1M → 1 hour, 1Y/ALL → 24 hours. Shorter ranges refresh more often because they're more time-sensitive.

9.6 AI portfolio analysis (AiAdvisorService.analyzePortfolio)

1. Load the user's active holdings + current prices.
2. Compute total value, per-holding allocation %, and a sector for each symbol.
3. Build a big text prompt describing the portfolio + an exact JSON schema to return.
4. POST to Gemini (gemini-2.5-flash).
5. Extract the model's text output, strip ```json fences, parse into AiAdvisorResponse.

The clever part is **asking the LLM to return strict JSON** matching the DTO, so the reply can be deserialized straight into a typed object.

10. Glossary of Terms & Definitions

Quick definitions of every important term in this project — useful when an interviewer asks “what is X?”.

- **SPA (Single-Page Application):** a web app that loads one HTML page and updates content with JavaScript instead of full page reloads. The React frontend is an SPA.
- **REST API:** an HTTP API where resources (users, portfolios, stocks) are accessed via URLs and standard verbs (GET/POST/DELETE). The backend is a REST API.
- **JWT (JSON Web Token):** a signed, self-contained token proving who the user is. Has three parts (header.payload.signature). Used here for stateless auth.
- **Stateless authentication:** the server keeps no session in memory; each request carries the JWT, which the server verifies. Scales well because any server instance can handle any request.
- **BCrypt:** a slow, salted password-hashing algorithm. We store the hash, never the raw password.
- **Spring Boot:** an opinionated framework on top of Spring that auto-configures a lot, so you can build a production REST service quickly.
- **Spring Data JPA:** lets you define repository *interfaces* and get CRUD + query methods automatically, backed by Hibernate.
- **JPA / Hibernate:** JPA is the Java persistence specification; Hibernate is the implementation that maps Java objects to DB tables (ORM).
- **ORM (Object-Relational Mapping):** mapping between Java objects (entities) and relational tables so you work with objects instead of SQL.
- **Entity:** a Java class mapped to a DB table (@Entity).
- **DTO (Data Transfer Object):** a plain object used to move data across a boundary (API in/out) without exposing entities. Here they're Java records.
- **Repository:** the data-access layer; an interface extending JpaRepository.
- **Service layer:** holds business logic; sits between controllers and repositories.
- **Controller:** maps HTTP requests to method calls and returns responses.
- **@Transactional:** wraps a method in a DB transaction — all-or-nothing.
- **BigDecimal:** an exact decimal type in Java, used for money (never double, which has rounding errors).
- **Bean Validation (@Valid, @NotBlank, etc.):** declarative input validation on DTOs/entities.
- **CORS (Cross-Origin Resource Sharing):** browser security that blocks JS calls to a different origin unless the server allows it. We allow the Vite dev origin.
- **CSRF (Cross-Site Request Forgery):** an attack where another site makes authenticated requests using your cookies. Disabled here because we use header tokens, not cookies.
- **Interceptor (Axios):** a hook that runs before every request (used to attach the JWT) or after every response.

- **Context API (React):** React's built-in way to share state (auth, theme) without "prop drilling".
- **Hook:** a React function (useState, useEffect, custom useAuth) that adds state/behaviour to components.
- **Debounce:** waiting until the user stops typing (300 ms) before firing a search, to reduce API calls.
- **TTL (Time To Live):** how long cached data is considered fresh before refetching.
- **@Scheduled:** Spring annotation to run a method on a timer (used for the price refresher).
- **fixedDelay vs fixedRate:** fixedDelay waits N ms *after the previous run finishes*; fixedRate starts every N ms regardless. We use fixedDelay to avoid overlaps.
- **Soft delete:** marking a row inactive (active = false) instead of physically deleting it.
- **Pagination:** returning data one "page" at a time (page, size) instead of all rows.
- **Projection (Spring Data):** an interface that maps only selected query columns (used for valuation totals).
- **RestClient:** Spring's modern synchronous HTTP client used to call Finnhub/Twelve Data/Gemini.
- **Weighted average:** an average that accounts for quantities — used for average buy price.
- **P/L (Profit / Loss):** (currentPrice – averageBuyPrice) × quantity.

11. Interview Questions — Conceptual / Definitions

These test whether you understand the fundamentals behind the tech you used.

Q1. What is Spring Boot and why did you use it? Spring Boot is a framework that makes it fast to build production-ready Spring applications. It auto-configures common things (an embedded Tomcat server, JSON handling, JPA), so I could focus on business logic instead of boilerplate XML/config. I used it because it's the industry standard for Java REST APIs and integrates cleanly with Spring Security and Spring Data JPA, both of which I needed.

Q2. What is the difference between Spring and Spring Boot? Spring is the core framework (dependency injection, MVC, etc.) but needs a lot of manual configuration. Spring Boot sits on top and provides auto-configuration, starter dependencies, and an embedded server so an app "just runs" with main().

Q3. What is dependency injection? Where is it used here? Dependency injection means the framework creates and supplies an object's dependencies instead of the object creating them itself. In this project, controllers receive services, and services receive repositories, via constructor injection (Lombok's @RequiredArgsConstructor). This makes the code loosely coupled and easy to test.

Q4. What is a REST API? An API that exposes resources over HTTP using URLs and standard methods — GET to read, POST to create, DELETE to remove — and typically exchanges JSON. My backend is a REST API; e.g. GET /api/portfolio reads holdings, POST /api/portfolio/buy creates a trade.

Q5. What is JPA and how is it different from Hibernate? JPA (Jakarta Persistence API) is a *specification* for object-relational mapping in Java. Hibernate is the most common *implementation* of that spec. I use Spring Data JPA, which sits on top of Hibernate and generates repository implementations for me.

Q6. What is an ORM? What problem does it solve? Object-Relational Mapping maps Java objects to database tables. It solves the "impedance mismatch" between objects and relational rows, so I write portfolioRepository.save(portfolio) instead of hand-writing SQL INSERTs and mapping result sets manually.

Q7. What is a DTO and why not just return the entity? A DTO is a Data Transfer Object — a plain object for moving data across a boundary. I return DTOs (Java records) instead of entities so I don't leak internal fields (like passwordHash), avoid lazy-loading issues during JSON serialization, and can shape the response exactly as the frontend needs.

Q8. What is JWT and how does it work? A JSON Web Token is a signed token with three parts: header, payload (claims like the subject/email and expiry), and a signature. The server signs it with a secret when you log in. On later requests the client sends it in the Authorization header; the server verifies the signature and expiry to trust the identity — no server-side session needed.

Q9. Why is JWT called “stateless”? Because the server stores nothing about the session. All the information needed to authenticate is inside the token itself, verified by signature. Any server instance can handle any request, which makes horizontal scaling easy.

Q10. What is BCrypt and why not store passwords directly or use MD5/SHA-256? BCrypt is a deliberately slow, salted hashing algorithm designed for passwords. Storing plaintext is catastrophic if the DB leaks. Fast hashes like MD5/SHA-256 can be brute-forced with billions of guesses per second; BCrypt’s cost factor makes that far slower, and its built-in salt defeats rainbow tables.

Q11. What is the difference between authentication and authorization? Authentication = proving *who you are* (login, JWT validation). Authorization = deciding *what you’re allowed to do* (e.g. `/api/admin/**` needs `ROLE_ADMIN`). My app does authentication via JWT and role-based authorization via Spring Security.

Q12. What is CORS and why did you configure it? CORS is a browser rule that blocks JavaScript from calling a different origin unless the server permits it. My frontend runs on `localhost:5173` and the backend on `localhost:8080` — different origins — so I configured `SecurityConfig` to allow that origin, the needed methods, and the Authorization header.

Q13. Why is CSRF disabled in this project? CSRF attacks rely on the browser automatically attaching credentials (cookies) to requests. My auth uses a JWT sent explicitly in a header, not an auto-sent cookie, so CSRF isn’t applicable — hence it’s safely disabled for a stateless API.

Q14. What is @Transactional and why is it important for buy/sell? It wraps a method in a single database transaction. Buying involves several writes (update holding, deduct cash, insert transaction). If any step fails, `@Transactional` rolls everything back so I never end up with, say, cash deducted but no shares added.

Q15. Why use BigDecimal instead of double for money? `double` is binary floating point and can’t represent many decimal values exactly ($0.1 + 0.2 \neq 0.3$). For money that causes rounding errors. `BigDecimal` is exact and lets me control scale (4 decimals) and rounding (`HALF_UP`).

Q16. What is the Context API in React and why use it here? It’s React’s built-in state-sharing mechanism. I used it for auth and theme so any component can read the token, cash balance, or dark-mode flag without passing props through every level (“prop drilling”). For a small app it’s simpler than Redux.

Q17. What is a React hook? Name the ones you used. A hook is a function that lets a component use state and lifecycle features. I used `useState`, `useEffect`, `useMemo`, `useCallback`, `useContext`, `useRef`, plus custom hooks `useAuth`, `useTheme`, and `useStockLogo`.

Q18. What is debouncing and where did you use it? Debouncing delays an action until the user stops triggering it. On the Buy/Sell page, the symbol search waits 300 ms after the last keystroke before calling the search API, which avoids firing a request on every character.

Q19. What is pagination and why does it matter? Returning data one page at a time (page, size) instead of everything at once. It keeps responses small and fast and avoids loading thousands of rows into the browser. My `GET /api/portfolio` is paginated.

Q20. What does fixedDelay mean for the scheduler and how is it different from fixedRate? `fixedDelay` schedules the next run a fixed time *after the previous run finishes*, so runs never overlap. `fixedRate` runs every N ms from each start time regardless of duration, which can overlap if a run is slow. I chose `fixedDelay` (50 min) so a slow price refresh can’t stack up.

12. Interview Questions — How This Project Works

These test whether you truly understand *your own* project’s behaviour.

Q1. Walk me through what happens when a user logs in. The `LoginPage` form calls `login()` from `useAuth`, which calls `POST /api/auth/login`. The backend’s `AuthService.login` uses the `AuthenticationManager` to verify the email and BCrypt password. On success it generates a JWT (subject = email, 24h expiry) and returns an `AuthResponse`. The frontend saves the token in `localStorage`, sets it in auth context, and navigates to the dashboard (or the page the user originally wanted).

Q2. How does the app know a user is logged in on later requests? Every Axios request runs a request interceptor that reads the JWT from `localStorage` and adds `Authorization: Bearer <token>`. On the backend the `JwtAuthenticationFilter` validates that token on each request and sets the authenticated user into the `SecurityContext`, so controllers know who's calling.

Q3. What exactly happens when a user buys a stock? The frontend posts `{symbol, quantity}` to `/api/portfolio/buy`. `PortfolioService.buyStock` fetches a live price from `Finnhub`, checks the user has enough cash ($\text{cost} = \text{qty} \times \text{price}$), finds or creates the holding, recomputes the weighted-average buy price, deducts the cash, saves the holding, and records a BUY transaction — all in one transaction. It returns the updated holding.

Q4. Where does the money come from? Is it real? No. Every user starts with a virtual `cash_balance` of \$100,000 stored on their user row. Buying subtracts from it and selling adds to it. Only the *prices* are real (from `Finnhub/Twelve Data`).

Q5. How is a stock's current price obtained? Two ways: on demand (a quote fetch or a buy/sell hits `Finnhub` live and saves the price to `stock_prices`), and in the background (a scheduled job every ~50 min refreshes prices for all tracked symbols). Valuations read the latest saved price.

Q6. How is profit/loss calculated? $\text{profitLoss} = (\text{currentPrice} - \text{averageBuyPrice}) \times \text{quantity}$. For the portfolio list this is computed in SQL by joining `stock_prices` on the symbol, with `COALESCE` to fall back to the average buy price if no live price exists.

Q7. How does the average buy price update when I buy more shares? It's a weighted average: $(\text{oldQty} \times \text{oldAvg} + \text{newQty} \times \text{newPrice}) / (\text{oldQty} + \text{newQty})$. So buying 10 @ \$100 then 10 @ \$120 gives an average of \$110.

Q8. What happens when you sell all your shares of a stock? The holding's quantity goes to zero, `active` is set to `false` (soft delete), and the average buy price is reset to 0. The row stays in the DB for history but no longer shows in the active portfolio.

Q9. How does the historical price chart work? The frontend calls `GET /api/stocks/history/{symbol}?range=...`. The backend checks the `stock_history_cache` table; if a fresh cached series exists (within its TTL) it returns that, otherwise it fetches from `Twelve Data`, formats the points, caches them as JSON, and returns them. `Recharts` renders the area chart.

Q10. How does the AI advisor feature work? `GET /api/portfolio/analyze` builds a text prompt describing the user's holdings, allocations, and sectors, and asks `Google Gemini` to return a strict JSON analysis (scores, risk level, strengths, weaknesses, per-stock recommendations). The backend parses that JSON into `AiAdvisorResponse` and the `AiAdvisorModal` displays it with gauges and badges.

Q11. How does dark mode work? `ThemeProvider` stores the theme in `localStorage` (defaulting to the OS preference) and toggles a `.dark` class on the `<html>` element. `Tailwind v4`'s class-based dark variant then applies dark styles. The header has a toggle button wired to `toggleTheme`.

Q12. How does routing and route protection work? `React Router` defines routes inside one `AppLayout`. Protected routes are wrapped in `ProtectedRoute`, which checks `isAuthenticated` from auth context; if `false` it redirects to `/login` and remembers where the user was trying to go, so they land back there after logging in.

Q13. What is stored in the JWT payload? Only the subject (the user's email) and standard timestamps (`issued-at`, `expiry`). No roles or IDs. The backend re-loads the user and their role from the DB on each request.

Q14. Why does buying a stock also save its price to the database? Because the buy already made a live API call to get the execution price, so saving it updates the shared `stock_prices` row for free and makes that symbol "tracked" so the scheduler keeps refreshing it later.

Q15. What does the scheduled job actually do? `StockPriceUpdateScheduler` runs 10 minutes after startup and then every 50 minutes. It lists every symbol in `stock_prices` and refetches each one's latest price from `Finnhub`, catching errors per symbol so one failure doesn't stop the rest.

Q16. How is input validated? On DTOs with Bean Validation annotations (`@NotBlank`, `@Email`, `@Size`, `@Pattern`, `@DecimalMin`) triggered by `@Valid` in controllers. The symbol is also re-validated in

the service with a regex. Validation failures return HTTP 400 with a field→message map.

Q17. What happens if an external API (Finnhub/Twelve Data) is down or rate-limited? StockApiService maps HTTP 429 to StockApiRateLimitException (returned as 429) and other errors to StockApiException (returned as 502). The scheduler logs and skips failing symbols; user-facing calls surface a clean error message.

Q18. How does the portfolio table sorting work? The frontend sends a sort parameter (e.g. holdingValue,desc). The backend validates it against a whitelist of allowed fields and builds the ORDER BY in the JPQL query using CASE expressions, so sorting happens in the database on the correct computed columns.

Q19. Where is the JWT stored on the client and why does that matter? In localStorage. It's convenient (survives refreshes, easy to read in the interceptor) but it's accessible to JavaScript, so it's vulnerable to XSS. A more secure option is an httpOnly cookie. I'd mention this as a known trade-off.

Q20. What happens if the token expires while I'm using the app? The next protected request fails (the filter returns 401). Currently there's no auto-refresh, so the user is effectively logged out and must sign in again. Adding refresh tokens or a 401 response interceptor would improve this.

13. Interview Questions — How Things Connect

These probe the “wiring” between components and layers.

Q1. How do the frontend and backend communicate? Over HTTP with JSON. The React app uses an Axios client pointed at http://localhost:8080; the Spring Boot app exposes REST endpoints. Auth is carried in the Authorization: Bearer header.

Q2. Trace the layers a request passes through on the backend. Security filter chain (including JwtAuthenticationFilter) → Controller → Service → Repository → PostgreSQL. The response travels back the same way, serialized to JSON by Spring/Jackson. Exceptions are intercepted by the GlobalExceptionHandler.

Q3. How does the controller know which user is making the request? The JWT filter sets an Authentication in the SecurityContext. Spring injects that Authentication into the controller method, and authentication.getName() returns the email (the token's subject), which is passed to the service to scope all queries to that user.

Q4. How do services talk to the database? Through Spring Data JPA repository interfaces. I call methods like portfolioRepository.findByUserEmailAndStockSymbol(...); Spring generates the SQL. For valuation I wrote custom @Query JPQL.

Q5. How is the frontend auth state connected to actual HTTP calls? AuthProvider keeps the token in state and localStorage. The Axios request interceptor reads the token from localStorage on every call (not from React state), so it's always in sync even right after login.

Q6. How do the three external APIs plug into the backend? StockApiConfig defines a separate RestClient bean for each (Finnhub, Twelve Data, Gemini) with the right base URL, auth header/param, and timeouts. Services inject the bean they need and call it.

Q7. How is the database connected to the entities? Via JPA annotations (@Entity, @Table, @Column, @ManyToOne, @OneToMany). Hibernate maps classes to tables. spring.jpa.hibernate.ddl-auto=update lets Hibernate create/adjust tables from the entities in dev.

Q8. How are Portfolio and StockPrice related if there's no foreign key between them? They're intentionally not a JPA relationship. Prices are global (shared by all users), so the valuation query joins them by the stock_symbol string in JPQL. This keeps prices decoupled from any single user's holdings.

Q9. How does the AI modal connect to Gemini? Frontend modal → GET /api/portfolio/analyze → AiAdvisorController → AiAdvisorService builds a prompt → geminiRestClient POSTs to the Gemini API → response parsed into a DTO → JSON back to the modal.

Q10. How do the buy/sell forms connect to the portfolio view? After a successful trade, the Buy/Sell page reloads the portfolio summary (and syncs cashBalance in auth context), so the header balance and any visible holdings reflect the change immediately.

Q11. How does React Router connect pages to URLs and to the layout? AppRoutes declares <Route>s as children of a layout <Route element={<AppLayout />}>. The layout renders <Outlet /> where the matched page appears. ProtectedRoute is an intermediate route element that gates the protected children.

Q12. How does the request interceptor connect to token storage? The interceptor imports getToken() from tokenStorage.js, which reads localStorage. So the single source of truth for the token is localStorage, and both the auth context and the interceptor read from it.

Q13. How does the scheduler connect to the price service and the DB? The scheduler injects StockPriceService, calls getTrackedSymbols() (which reads stock_prices), then fetchAndSaveLatestPrice() per symbol (which calls Finnhub and upserts stock_prices).

Q14. How does exception handling connect across the app? Services throw typed custom exceptions; the @RestControllerAdvice catches them centrally and converts each to the right HTTP status + ApiError JSON. The frontend's getErrorMessage reads error.response.data.message to show it.

Q15. How does pagination data flow from DB to UI? The repository returns a Page<Portfolio>; the service maps it to PortfolioValuationResponse with page, totalPages, totalElements; the frontend passes those to PaginationControls, whose Prev/Next buttons call setPage, triggering a refetch.

14. Interview Questions — Why This, Not That (Design Decisions)

These test your judgement. The trick is to give the trade-off, not just the choice.

Q1. Why JWT instead of server-side sessions? JWT is stateless — no session store to maintain, and it scales horizontally because any instance can validate the token. Sessions need sticky sessions or a shared store (Redis). Trade-off: you can't easily invalidate a JWT before it expires, and it's slightly bigger per request. For a small stateless API, JWT was the simpler fit.

Q2. Why React Context instead of Redux? The app only shares a little global state (auth token, cash balance, theme). Context covers that with zero extra dependencies and less boilerplate. Redux shines when you have lots of complex, frequently-updated shared state — overkill here. If the app grew, I'd consider Redux Toolkit or React Query.

Q3. Why PostgreSQL instead of MongoDB? The data is highly relational and transactional — users own holdings, holdings have transactions, and money movements must be consistent. A relational DB with ACID transactions fits naturally. MongoDB suits flexible/denormalized documents; here I want joins, constraints (unique user+symbol), and @Transactional safety.

Q4. Why BigDecimal instead of double? Money must be exact. double introduces binary rounding errors. BigDecimal gives exact arithmetic with explicit scale and rounding — essential for cash and P/L.

Q5. Why compute valuation in SQL instead of in Java? Doing holdingValue/profitLoss, sorting, and totals in the query means the DB only returns one page of already-sorted rows. Doing it in Java would require loading all holdings and all prices into memory and sorting there — slower and more memory. SQL is built for this.

Q6. Why RestClient instead of RestTemplate or WebClient? RestClient is Spring's modern synchronous client (replacing the now-maintenance-mode RestTemplate) with a clean fluent API and easy status-based error handling. WebClient is reactive/async, which I didn't need since the backend is a standard blocking MVC app.

Q7. Why store the JWT in localStorage and what's the downside? It's simple and survives page refreshes, and it's easy to read in the Axios interceptor. Downside: it's readable by JavaScript,

so an XSS bug could steal it. A more secure alternative is an `httpOnly` cookie (not JS-accessible) — at the cost of needing CSRF protection. I'd flag this as a known trade-off.

Q8. Why DTOs (records) instead of returning entities? To decouple the API from the DB schema, hide sensitive fields (password hash), avoid Hibernate lazy-loading serialization issues, and shape responses for the frontend. Records make immutable DTOs concise.

Q9. Why soft delete instead of hard delete for holdings? Soft delete (`active = false`) preserves history and the transaction audit trail, and avoids breaking references. It also lets a symbol be "reactivated" if the user buys it again. Hard delete would lose that history.

Q10. Why cache historical prices in the database and not in memory? A DB cache is shared across all users and survives restarts, and the data (a time series) is a natural fit for a text/JSON column keyed by (symbol, range). An in-memory cache would be per-instance and lost on restart. For live quotes, freshness matters more, so those aren't cached the same way.

Q11. Why `fixedDelay` instead of `fixedRate` for the scheduler? `fixedDelay` guarantees the next run starts only after the previous finishes, so slow runs (many symbols, slow API) can't overlap and pile up. `fixedRate` could launch overlapping runs.

Q12. Why re-load roles from the DB instead of putting them in the JWT? Keeping the token minimal (just the email) means role/permission changes take effect immediately on the next request, instead of being "frozen" in a token until it expires. The cost is one DB lookup per request, which is cheap.

Q13. Why Vite instead of Create React App? Vite has a much faster dev server (native ES modules + esbuild) and quicker builds, and CRA is effectively deprecated. Vite is the current default for new React projects.

Q14. Why Tailwind instead of plain CSS or a component library? Tailwind's utility classes let me style quickly and consistently without writing lots of custom CSS or context-switching between files. Trade-off: markup gets verbose. For a solo project with a custom look, it was efficient.

Q15. Why separate the frontend and backend into two apps? Separation of concerns: they can be developed, deployed, and scaled independently, and the same backend could serve a mobile app later. Trade-off: you must handle CORS and can't share code as easily as a monolith.

Q16. Why weighted-average cost instead of FIFO/LIFO for cost basis? Weighted average is simpler to implement and understand and needs only one number per holding. FIFO/LIFO require tracking individual lots. For a learning/paper-trading app, average cost is a reasonable, clear choice (real tax reporting might need FIFO).

Q17. Why validate the symbol both at the DTO and in the service? Defense in depth. The DTO `@Pattern` gives a fast, declarative 400 for obviously bad input; the service re-check protects any code path that doesn't go through that DTO and normalizes the symbol (trim/upper) before use.

Q18. Why fetch a live price on every buy/sell instead of using the cached one? Trades must execute at a realistic current price, not a possibly-stale cached value. So buy/sell always hit the live quote. The trade-off is external-API dependency on the critical path (discussed in situational questions).

15. Interview Questions — Situational ("What would you do if...")

These test how you'd evolve the system. Structure your answer as: *problem* → *approach* → *trade-off*.

Q1. What if the app suddenly had 1 million users? I'd (a) run multiple stateless backend instances behind a load balancer — easy because JWT auth is stateless; (b) add DB connection pooling and read replicas; (c) introduce caching (Redis) for hot data like prices; (d) move external-API calls off the request path where possible; (e) add indexes (already have some on `user_id`/symbol) and monitor slow queries. I'd load-test to find the real bottleneck before optimizing blindly.

Q2. What if Finnhub goes down during a buy? Right now the buy fails with a 502 because the live price can't be fetched. To harden it I'd add a retry with backoff, a fallback to the last known price

with a “stale price” flag, or a circuit breaker (Resilience4j) so we fail fast and show a clear message instead of hanging. I’d also consider queuing the trade for later execution.

Q3. What if you hit the external API rate limit? The code already maps 429 to a `StockApi-RateLimitException`. To reduce hitting it: cache quotes for a few seconds, batch symbol lookups, add client-side throttling/backoff, and spread the scheduler’s calls out. Long term, a paid tier or a message queue to smooth bursts.

Q4. What if two requests try to buy the same stock for the same user at the same time? That’s a race condition on the holding row and cash balance. I’d use optimistic locking (a JPA `@Version` column) so the second commit fails and retries, or pessimistic locking (`SELECT ... FOR UPDATE`) on the holding/user row inside the transaction. `@Transactional` alone doesn’t prevent lost updates from concurrent transactions.

Q5. How would you add refresh tokens? Issue a short-lived access token (e.g. 15 min) plus a long-lived refresh token stored securely (httpOnly cookie or DB). Add a `/api/auth/refresh` endpoint. On the frontend, a response interceptor catches 401, calls refresh, retries the original request. This improves security and UX (no sudden logouts).

Q6. How would you support real-time price updates on the UI? Instead of polling, use WebSockets (Spring’s `STOMP/@MessageMapping`) or Server-Sent Events to push price changes to subscribed clients. The scheduler (or a streaming price feed) would broadcast updates, and the React charts would update live.

Q7. How would you make the AI analysis faster / cheaper? Cache the analysis per user for a short window (portfolios don’t change every second), stream the response, use a smaller/faster model for simple portfolios, and run the call asynchronously with a “generating...” state (the UI already fakes this). I’d also add a timeout and graceful fallback.

Q8. What if you needed to support multiple currencies? Store a currency code per holding/price, add an exchange-rate service (with caching), and convert to the user’s base currency for display and totals. I’d keep amounts in their native currency in the DB and convert at read time to avoid rounding drift.

Q9. How would you add automated tests to this project? Unit tests for services (mock repositories and the `RestClients`) covering buy/sell edge cases and the average-price maths; `@DataJpaTest` for the custom repository queries; `@WebMvcTest/MockMvc` for controllers and security rules; and a few integration tests with `Testcontainers` (a real PostgreSQL). The test starters are already in the pom.

Q10. How would you deploy this to production? Containerize both apps with Docker; serve the React build behind a CDN or Nginx; run the Spring Boot app behind a load balancer; use a managed PostgreSQL; inject secrets via environment variables/secret manager (never the committed defaults); set `ddl-auto=validate` and use Flyway/Liquibase for migrations; enable HTTPS and set the real CORS origin.

Q11. What if a user reports their portfolio total looks wrong? I’d check whether `stock_prices` has a fresh price for the symbols (else it falls back to average buy price, making P/L read 0), verify the scheduler is running, look at the valuation query’s `COALESCE` logic, and confirm the frontend is reading the right fields. The `portfolio_transactions` audit log lets me reconstruct the expected state.

Q12. How would you prevent a user from seeing another user’s portfolio? Every query is scoped by the authenticated email (`findByUserEmail...`), and the user comes from the verified JWT, not from a request parameter — so a user can’t request someone else’s data by changing an ID. For the DELETE-by-id endpoint, the query is `findByIdAndUserEmailAndActiveTrue`, so you can only delete your own holdings.

Q13. What if the scheduled job takes longer than its interval? Because it uses `fixedDelay`, the next run simply starts 50 min after the current one *finishes*, so it can’t overlap itself. If runs got very long I’d parallelize per-symbol fetches with a bounded thread pool or split symbols across workers.

Q14. How would you add a “watchlist” feature properly (persisted)? Add a `watchlist` table (`user_id`, `symbol`), endpoints to add/remove/list, and have the scheduler also track watchlist symbols. The frontend already has a client-side watchlist chart; I’d back it with this table so it persists per user.

Q15. What if you needed an audit of who changed what and when? BaseEntity already stamps createdAt/updatedAt, and portfolio_transactions logs every trade. For fuller auditing I'd add Hibernate Envers or an append-only audit table capturing the actor, action, before/after, and timestamp.

Q16. How would you handle fractional shares vs whole shares? The schema already uses DECIMAL(19,4) for quantity, so fractional shares are supported. If the business required whole shares only, I'd add a validation rule (quantity must be an integer) at the DTO and service level.

Q17. What if the frontend and backend must be on the same domain to avoid CORS? Serve the built React files from the Spring Boot app (as static resources) or put both behind one reverse proxy (Nginx) with path routing (/api → backend, everything else → frontend). Then requests are same-origin and CORS isn't needed.

Q18. How would you secure the committed API keys and DB password? Remove the defaults, load everything from environment variables or a secrets manager (Vault, AWS Secrets Manager), and rotate the exposed keys immediately. Never commit real secrets; use a .env that's git-ignored plus example templates.

16. Interview Questions — Coding / Hands-on Drill-downs

Expect to be asked to write or read small pieces of code, or explain specific snippets.

Q1. Write the weighted-average buy price calculation.

```
BigDecimal newAvg = existingQty.multiply(existingAvg)
    .add(buyQty.multiply(buyPrice))
    .divide(existingQty.add(buyQty), 4, RoundingMode.HALF_UP);
```

Key points: use BigDecimal methods (multiply, add, divide), and always pass a scale + Rounding-Mode to divide or it throws on non-terminating decimals.

Q2. How do you define a Spring Data repository method to find a holding by user email and symbol?

```
Optional<Portfolio> findByUserEmailAndStockSymbol(String email, String stockSymbol);
```

Spring parses the method name and generates the query, traversing Portfolio.user.email.

Q3. How is an endpoint secured to a logged-in user only? It's not annotated per-method here; SecurityConfig says anyRequest().authenticated(), and public paths are explicitly permitAll. The current user is obtained via the injected Authentication:

```
@PostMapping("/buy")
public PortfolioHoldingResponse buy(@Valid @RequestBody PortfolioTradeRequest req,
    Authentication auth) {
    return portfolioService.buyStock(auth.getName(), req);
}
```

Q4. Show a DTO with validation.

```
public record PortfolioTradeRequest(
    @NotBlank @Pattern(regexp = "[A-Za-z0-9.-]{1,20}$") String symbol,
    @NotNull @DecimalMin("0.0001") BigDecimal quantity) {}
```

Q5. How do you attach the JWT to every Axios request?

```
apiClient.interceptors.request.use((config) => {
    const token = getToken();
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
```

Q6. How do you generate a JWT with JJWT 0.12.x?

```
return Jwts.builder()
    .subject(userDetails.getUsername())
    .issuedAt(now)
    .expiration(new Date(now.getTime() + expirationMs))
    .signWith(getSigningKey())
    .compact();
```

Q7. Explain @OneToMany / @ManyToOne mapping between User and Portfolio.

```
// User
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Portfolio> portfolios;

// Portfolio
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id")
private User user;
```

mappedBy says the Portfolio.user field owns the FK. LAZY avoids loading the user until accessed.

Q8. Write a custom hook that reads auth context.

```
export function useAuth() {
    const ctx = useContext(AuthContext);
    if (!ctx) throw new Error("useAuth must be used within AuthProvider");
    return ctx;
}
```

Q9. How do you make a component re-render when a value changes but avoid recomputing expensive work? Use useMemo for derived values and useCallback for stable function references. Example: the portfolio table memoizes its sorted/filtered rows so it only re-sorts when the data or sort option changes.

Q10. How would you handle an error from an API call in React?

```
try {
    await buyStock({ symbol, quantity });
} catch (err) {
    setError(err.response?.data?.message || "Trade failed");
}
```

The backend returns {message} in the ApiError body, which getErrorMessage reads.

Q11. What does @Transactional(readOnly = true) do and where is it used? It marks the transaction as read-only, letting the DB/Hibernate optimize (no dirty-checking/flush). Used on getPortfolio, analyzePortfolio, and getTrackedSymbols, which only read.

Q12. How do you return the right HTTP status for a custom exception?

```
@ExceptionHandler(InsufficientCashBalanceException.class)
public ResponseEntity<ApiError> handle(InsufficientCashBalanceException ex) {
    return ResponseEntity.badRequest()
        .body(new ApiError(400, ex.getMessage(), Instant.now()));
}
```

Q13. How does the backend return paginated data? The controller accepts a Pageable (@PageableDefault), the repository returns a Page<Portfolio>, and the service copies getNumber(), getTotalPages(), getTotalElements() into the response DTO along with the mapped holdings.

17. Interview Questions — Behavioural / Project-story

These are the “tell me about your project” questions. Have crisp, confident answers ready.

Q1. Tell me about this project in 60 seconds. “Stock Portfolio Manager is a full-stack paper-trading app. The backend is a Spring Boot REST API with JWT security, JPA/PostgreSQL, and integrations with Finnhub and Twelve Data for real market data, plus Google Gemini for AI portfolio analysis. The frontend is a React + Vite SPA with charts, dark mode, and an AI advisor. Users get \$100k virtual cash, buy and sell real symbols at live prices, and track profit/loss. I built it to practice production-style layering, security, and third-party integration.”

Q2. What was the hardest part and how did you solve it? “Getting portfolio valuation right and efficient. Computing holding value and P/L for a sortable, paginated list meant joining live prices to holdings. I moved that logic into a JPQL query with a LEFT JOIN on the price table and a dynamic ORDER BY, so the database does the maths and sorting and I only return one page — instead of loading everything into memory.”

Q3. What did you learn from this project? “How the layers of a real Spring app fit together (controller/service/repository), how JWT stateless auth actually works end to end, why money needs BigDecimal, and the practical realities of depending on third-party APIs — timeouts, rate limits, and caching.”

Q4. What would you do differently if you started over? “I’d add tests from the start, keep secrets out of the repo from day one, and design the price fetching so external calls aren’t on the critical path of a trade. I’d also plan for refresh tokens and put the token in an httpOnly cookie.”

Q5. What are you most proud of? “The clean layering and the fact that risky operations like buying are transactional and validated at multiple levels, plus the caching strategy for historical data that keeps the app responsive without hammering the APIs.”

Q6. Did you work on this alone? How did you plan it? “Yes. I started from the data model (users, holdings, transactions, prices), built the backend layer by layer with security first, then built the frontend page by page against those endpoints, adding charts and the AI feature last.”

Q7. How did you test it manually? “I used the API directly (Postman/curl) to verify auth, buy/sell edge cases (insufficient cash, selling more than owned), and error responses, then verified the same flows through the UI, including token expiry and protected-route redirects.”

Q8. How do you keep the code maintainable? “Strict layering with single responsibilities, DTOs to isolate the API from the schema, centralized exception handling, typed custom exceptions, and consistent money handling. On the frontend, small components, custom hooks, and a single API client.”

18. Known Limitations & How to Talk About Them

Interviewers respect candidates who know their project’s weak spots. Frame each as “I know this, and here’s the fix.”

- **JWT in localStorage** — XSS-exposed. Fix: httpOnly cookie + CSRF protection, or at least strict input sanitization/CSP.
- **Secrets committed as defaults** in application.properties. Fix: environment variables / secret manager; rotate keys.
- **No refresh tokens** — users are logged out abruptly at 24h. Fix: short access token + refresh flow + 401 interceptor.
- **External APIs on the trade critical path** — a Finnhub outage blocks buys. Fix: retries, circuit breaker, fallback price.
- **No automated tests** despite test starters being present. Fix: unit + slice + integration tests (Testcontainers).
- **No concurrency control** on simultaneous trades for the same user. Fix: optimistic locking (@Version) or row locks.

- **Hardcoded frontend base URL** (localhost:8080). Fix: VITE_API_URL env variable per environment.
- **Some placeholder/unused pieces** (a hardcoded “market summary”, unused demo components). Fix: wire to real data or remove.
- **ddl-auto=update in dev** is convenient but risky for prod. Fix: switch to validate + Flyway/Liquibase migrations.
- **/api/admin/** rule exists with no admin controller** — harmless but dead config. Fix: implement or remove.

19. Quick Revision Cheat-Sheet

One-liner: Full-stack paper-trading app — Spring Boot + PostgreSQL backend, React + Vite frontend, JWT auth, live prices (Finnhub/Twelve Data), AI analysis (Gemini). Each user starts with \$100k virtual cash.

Backend layering: Controller → Service → Repository → PostgreSQL, with a JWT security filter in front and a global exception handler around it.

Auth: Stateless JWT (HS256, 24h, subject = email). BCrypt passwords. Token in Authorization: Bearer. Roles loaded from DB per request.

Money rules: BigDecimal(19,4), HALF_UP. Cash on the user row. Buy = deduct cash + weighted-avg price. Sell = add cash; sell-to-zero soft-deletes.

Valuation: holdingValue = qty × price; P/L = (price − avgBuy) × qty; computed & sorted in SQL, paginated.

Caching: History cached in DB per (symbol, range) with TTL (1D=5m, 1W/1M=1h, 1Y/ALL=24h). Prices refreshed by a @Scheduled job every 50 min (fixedDelay).

Frontend: React 19, Router 7, Axios (request interceptor adds token), Recharts, Tailwind 4 (class-based dark mode), Context API for auth/theme, protected routes, debounced search, lazy-loaded charts.

Key endpoints: POST /api/auth/{register,login}, POST /api/portfolio/{buy,sell}, GET /api/portfolio, GET /api/portfolio/analyze, GET /api/stocks/{search,{symbol},history/{symbol}}.

Top trade-offs to mention: JWT vs sessions; Context vs Redux; SQL valuation vs Java; RestClient vs WebClient; BigDecimal vs double; soft vs hard delete; localStorage vs httpOnly cookie.

Top improvements to mention: tests, refresh tokens, secrets management, circuit breaker for external APIs, concurrency locking, WebSocket live prices, DB migrations.

30-second definitions to nail: JWT, stateless auth, BCrypt, DTO, ORM/JPA, @Transactional, CORS vs CSRF, dependency injection, pagination, debounce, TTL, weighted average.

End of guide. Good luck with the interview — read §12 and §15 last before you walk in.